

Assignment 4 Report: Sharding Design for Multi-Course Attendance Management Portal

Your Name
Indian Institute of Technology Gandhinagar
Database Systems Assignment 4

Abstract—This report documents the sharding design process for the attendance management workload in Module-B. The current dataset includes approximately 60 students, 10 courses, around 120 attendance sessions, and around 2400 attendance records. This submission covers SubTask 1 (first part): a complete list of candidate shard keys before selecting the final key.

I. SUBTASK 1: SHARD KEY SELECTION AND JUSTIFICATION

A. Candidate Shard Keys Considered

The target high-volume table for horizontal partitioning is `attendance_records`. A shard key should preserve locality for dominant queries, support balanced growth, and reduce cross-shard joins. Before finalizing one key, all realistic candidates were enumerated:

TABLE I
CANDIDATE SHARD KEYS FOR `ATTENDANCE_RECORDS`

Candidate Key	Type	Why It Is a Candidate
<code>record_id</code>	Direct column (PK)	Unique, monotonically increasing, simple range/hash partitioning.
<code>student_id</code>	Direct column (FK)	Most student-facing reads and updates filter by student. Strong entity locality for attendance history.
<code>att_session_id</code>	Direct column (FK)	Instructor/TA workflows often fetch or update all records for one session.
<code>course_id</code>	Derived via join	Many operational queries are course-scoped (attendance sessions, corrections, rosters).
<code>semester_id</code>	Derived via joins	Archive/report workloads are commonly semester-bounded.
<code>status</code>	Direct column	Correction workflows filter absent records; potentially useful as a secondary partition dimension.
<code>(student_id, course_id)</code>	Composite (derived)	Captures student-course locality, useful for per-course student analytics.
<code>(student_id, att_session_id)</code>	Composite (natural uniqueness)	Mirrors logical grain of a student attendance mark for one session.
<code>(course_id, att_session_id)</code>	Composite (derived+direct)	Preserves locality for session-level and course-level instructor operations.
<code>department</code>	Derived via profile joins	Can model organizational shards for dean/admin views.

B. Candidate Notes Aligned to Current Project

- **Current query patterns:** Student APIs are largely student-centric, while instructor and TA APIs are session/course-centric. Therefore both `student_id` and `att_session_id/course_id` are strong practical candidates.
- **Current scale:** With approximately 2400 attendance rows and approximately 60 students, student-scoped partitioning yields enough key cardinality for multiple shards while preserving read locality.
- **Implementation compatibility:** The migration script already demonstrates directory-based placement keyed by `student_id`, confirming low-friction adoption if this key is selected.

C. Measured Distribution Across Candidate Keys

Using the seeded dataset (2128 rows in `attendance_records`) and 3 target shards, candidate keys were evaluated by observed record counts per shard. Let the ideal per-shard load be

$$\mu = \frac{N}{S} = \frac{2128}{3} = 709.33$$

where N is total records and S is number of shards.

TABLE II
OBSERVED DISTRIBUTION AND DEVIATION (3 SHARDS)

Key	Counts	Max $ c_i - \mu $	Max Dev.
<code>student_id</code>	686, 736, 706	27.33	3.85%
<code>att_session_id</code>	691, 722, 715	18.67	2.63%
<code>record_id</code>	701, 706, 721	11.67	1.65%
<code>student_id+course_id</code>	688, 754, 686	44.67	6.30%
<code>semester_id</code>	1459, 669 (2 active shards)	749.67	105.69%
<code>status</code>	1797, 331 (2 values only)	1087.67	153.34%
<code>department</code>	999, 780, 349	360.33	50.80%

D. Immediate Elimination

Three candidates are rejected before final comparison:

- **`semester_id`:** severe skew (1459 vs 669) because only two semester values are active in the current dataset.
- **`status`:** only two values (`present/absent`), which cannot map cleanly to three shards and guarantees hotspots.

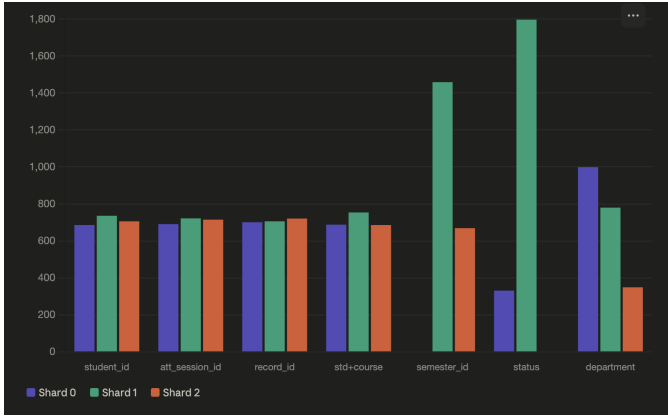


Fig. 1. Shard-wise record distribution for evaluated shard-key candidates.

- **department:** heavy organizational skew (999 vs 349), likely to persist as Computer Science dominates enrollment.

E. Viable Candidate Comparison

After elimination, the practical set is `record_id`, `att_session_id`, `student_id`, and composite `student_id+course_id`.

TABLE III
FINAL CANDIDATE COMPARISON

Key	Max Dev.	Join Req.?	Stable?	Query Aligned?
<code>record_id</code>	1.65%	No	Yes	No
<code>att_session_id</code>	2.63%	No	Yes	Partial
<code>student_id</code>	3.85%	No	Yes	Yes
<code>student_id+course_id</code>	6.30%	Yes	Yes	Yes

F. Candidate Evaluation Against the Three Criteria

1) *Criterion 1: High Cardinality:* Distinct value counts were measured from the current dataset:

TABLE IV
CARDINALITY OF CANDIDATE KEYS

Key	Distinct Values	Verdict
<code>record_id</code>	2128	Too high; PK with no semantic grouping
<code>att_session_id</code>	95	Moderate
<code>student_id</code>	60	Good; maps to real entities
<code>course_id</code>	10	Too low
<code>semester_id</code>	2	Eliminated
<code>department</code>	5	Too low
<code>status</code>	2	Eliminated

`student_id` has enough cardinality (60 distinct values) to support 3 shards while preserving student-level data locality. In contrast, `record_id` has very high cardinality but does not encode application-level access semantics.

2) *Criterion 2: Query Alignment:* Actual route logic is dominated by student-centric filters, with patterns such as:

- `WHERE ar.student_id = ?`
- `WHERE student_id = ? AND att_session_id = ?`
- `WHERE att.course_id = ? AND ar.student_id = ?`
- `WHERE student_id = ?`

TABLE V
QUERY-ALIGNMENT SUMMARY FOR FINALISTS

Key	Route Alignment	Impact
<code>record_id</code>	Low	Student history is scattered across shards
<code>att_session_id</code>	Partial	Helps instructor session views, weaker for student dashboards
<code>student_id</code>	High	Student-facing queries route to exactly one shard
<code>student_id+course_id</code>	High	Good locality but more complex routing logic

3) *Criterion 3: Stability:* Audit results show zero observed student identifier changes in the current dataset (`student_id_changes = 0`). Since `student_id` is assigned once and remains stable, shard ownership remains stable over time.

G. Selected Shard Key

Selected shard key: `student_id`

TABLE VI
FINAL DECISION MATRIX

Criterion	Result for <code>student_id</code>
High cardinality	60 distinct values (about 20 students per shard)
Query aligned	Present in core student-facing API predicates
Stable	0 observed key-change events in audit data

Why not `record_id` despite tighter balance? `record_id` is an auto-increment identifier with little query relevance. Choosing it would distribute each student's attendance history across multiple shards, forcing fan-out and merge behavior for common student operations.

H. Partitioning Strategy

The selected strategy is **range-based sharding on `student_id`**.

Why range-based for this project:

- **Predictable routing:** shard can be computed by simple boundary checks.
- **Low routing overhead:** no extra directory lookup per query.

- **Observed balance:** measured deviation remains low (about $\pm 3.8\%$).

Relative to alternatives used in evaluation, range-based `student_id` partitioning preserves student-locality while keeping shard loads near-even for current data volume.

I. Expected Distribution and Skew Risk

Using the current ranges and dataset:

TABLE VII
OBSERVED STUDENT-RANGE DISTRIBUTION (3 SHARDS)

Shard	<code>student_id</code> Range	Records	Deviation
Shard 0	12–30	686	-3.3%
Shard 1	31–50	736	+3.8%
Shard 2	51–71	706	-0.5%

Skew risks:

- **Enrollment growth drift:** newly created student IDs accumulate in the highest range, increasing load on the last shard.
- **Activity skew:** uneven attendance/correction activity by specific student groups can create write hotspots.
- **Lifecycle skew:** older ID cohorts may retain longer histories and heavier reads.

Mitigation plan: periodically re-evaluate boundaries and, if scale increases substantially, transition to hash-based or directory-based placement for smoother long-term balancing.